Recap of decoding process in Transformer model
00:12
Transformer generates next tokens
Auto regressive process during training
✦
Next token prediction can generate an entire story iteratively
01:18
Trained transformer used for next token prediction
Instruction F training involves generating specific stories like 'Once Upon a Time'
✦
Decoding Strategies involve predicting next tokens till end of sequence
02:19
Use special end of sequence marker to signify completion of answer generation
Implement instruction fine tuning for model to continue answers from certain input
✦
Decoding strategies involve generating tokens based on initial input
03:06
Pre-training and fine-tuning models on specific tasks are simpler compared to modern LLM capabilities
Modern LLMs handle arbitrary creative tasks, not just academic NLP tasks
✦
Decoding process in NLP models
04:05
Decoding in NLP involves generating output based on input through specific prompts.
Decoding remains a complex process, with details of training models still unclear.
✦
Completing simple sequence tasks during pre-training.
04:58
Model predicts the next token based on previous tokens iteratively.
Aim is to avoid incoherent or repetitive responses and encourage variety.
✦
Max probability token leads to same sequence
05:46
Taking the maximum probability token at each time step leads to deterministic outputs.
The model lacks stochasticity, resulting in the same sequence even with repeated inputs.
✦
Decoding strategies ensure output variety
06:35
The deterministic strategies include exhaustive search, greedy search, beam search, and contrastive decoding.
Stochastic strategies like top K sampling and top P (nucleus sampling) help generate diverse outputs.

The main takeaways from the given context on decoding strategies for token generation and storytelling are as follows:

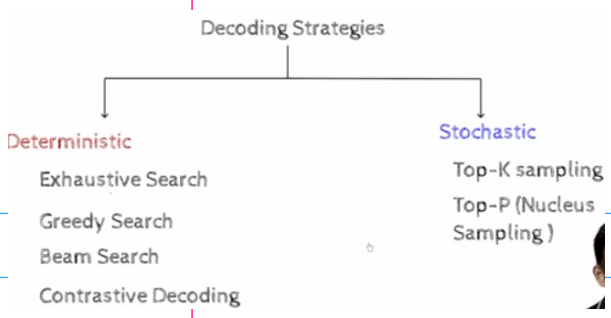
Decoding in Transformer model involves predicting next tokens until the end of the sequence.
The process includes generating tokens based on the initial input.
Next token prediction can be used to generate an entire story iteratively.
Decoding strategies ensure output variety and can be crucial in NLP models.
Completing simple sequence tasks during pre-training is essential for decoding processes.
The token with the maximum probability may lead to the same sequence being generated repeatedly.

In a Transformer model, tokens are generated through a process called decoding. During decoding, the model predicts the next token in a sequence based on the previously generated tokens. This prediction is typically done by calculating the probability distribution over the vocabulary of possible tokens and selecting the token with the highest probability as the predicted next token. This process is repeated iteratively to generate a sequence of tokens until a specific condition is met, such as reaching a maximum sequence length or generating a special end-of-sequence token. By using decoding strategies and techniques, Transformer models can effectively generate tokens for various tasks, including text generation and language translation.

The model calculates the probability distribution over the vocabulary of possible tokens using the softmax function. In the context of language modeling tasks, each token in the vocabulary is assigned a probability score based on the model's learned parameters and input sequence context.

The calculation involves applying a linear transformation to the model's hidden state representation corresponding to the input sequence, followed by the application of the softmax function. The softmax function normalizes the scores across all tokens in the vocabulary, producing a probability distribution where the sum of all probabilities equals one.

By selecting the token with the highest probability from this distribution, the model predicts the next token in the sequence. This process is repeated iteratively during decoding to generate a sequence of tokens. Through training on large datasets, the model learns to assign higher probabilities to tokens that are more likely to occur in a given context, leading to coherent and meaningful token generation.



Exhaustive search is a brute force approach used for academic purposes. It involves computing probabilities for sequences, assigning probabilities to sequences for language modeling, and calculating conditional probabilities using neural networks. It is a part of the deterministic decoding strategies employed in language model processes. However, while it is conceptually helpful for academic understanding, it is not recommended in practice due to its computational cost and inefficiency.

Sure! Let's consider a simple example to understand how exhaustive search can be used in word sequence generation. Suppose we have a language model designed to generate short sentences based on a given set of words. The model's goal is to find the most probable sequence of words given a specific context.

Using exhaustive search, the model would consider all possible combinations of words and calculate the probability of each sequence based on the language model's training data. For instance, if the context is "The cat", the model would consider all possible next words (e.g., "is", "sat", "runs", etc.) and calculate the probability of each complete sequence based on the language model's probabilities for word transitions.

Once the probabilities for all possible word sequences following "The cat" have been computed, the exhaustive search algorithm would select the sequence with the highest probability as the output. In this case, let's say the sequence "The cat sat" has the highest probability according to the language model's calculations, so it would be chosen as the generated output.

This example illustrates how exhaustive search systematically evaluates all possible word sequences given a context to determine the most probable outcome based on the language model's probabilities. However, it is important to note that while this approach ensures the selection of the most probable sequence, it can become computationally expensive as the number of possible sequences and the vocabulary size increase, making it impractical for real-time or large-scale text generation tasks.

The main takeaway from exhaustive search in deterministic algorithms for word sequence generation is that it is a comprehensive method for searching through all possible sequences to find the one with the highest probability according to a language model. This method calculates the conditional probabilities of different word sequences based on the context and selects the sequence with the highest probability. As an example, when generating text using an exhaustive search algorithm, the model considers all possible sequences of words given the context, calculates the probability of each sequence, and selects the one with the highest probability as the output sequence. While this approach ensures the selection of the most probable sequence, it is computationally expensive, especially for long sequences and large vocabularies. Therefore, it is more suitable for academic understanding of language modeling and decoding strategies rather than practical implementation due to its inefficiency in real-world applications.

In summary, exhaustive search provides a thorough understanding of how language models calculate probabilities for word sequences, but its impractical computational requirements make it unsuitable for real-world applications in text generation. Despite being a deterministic and comprehensive approach, it is not advisable for use in production systems due to its high computational cost and lack of efficiency.

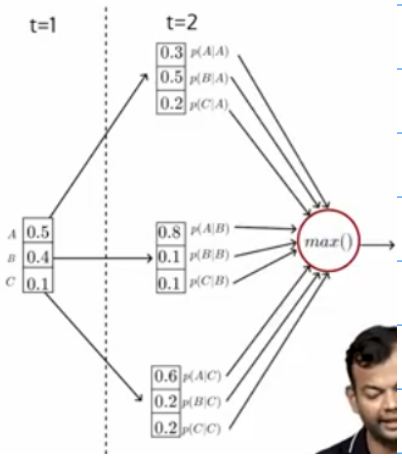
1/1

Illustration

Exhaustive search for a sequence of length 2 with the vocabulary of size 3

At time step-1, the decoder outputs probability for all 3 tokens

At time step-2, we need to run the decoder three times independently conditioned on all the three predictions from the previous time step



1. Greedy search selects tokens with maximum probability at each time step, leading to a straightforward but not necessarily optimal approach.
2. Beam search expands the search space by carrying forward the top outputs at each time step, balancing between maximum likelihood and the variety of sequences.
3. Beam search is ideal for deterministic outputs in tasks like translation and summarization.

An example of these takeaways could be illustrated in the context of machine translation. When translating a sentence from one language to another, the use of greedy search may lead to the selection of individual words based solely on their individual probabilities, potentially resulting in a translation that may not consider the overall context and structure of the sentence. On the other hand, beam search allows for the consideration of multiple possible translations by carrying forward the top sequences at each step, enabling a more context-aware and potentially more accurate translation while maintaining a balance between the likelihood of the translation and the diversity of possible outputs.

Certainly! Let's consider an example of using beam search for generating text sequences. Suppose we are using beam search to generate a caption for an image.

The image depicts a cat sitting on a windowsill. When using beam search to generate a caption for this image, the algorithm considers multiple possible sequences of words at each step while taking into account the probabilities of the words and their combinations.

At the first step, the algorithm may consider multiple starting words such as "A", "The", "In", "On", each with their respective probabilities based on the image features. These possibilities create branches in the search space.

As the algorithm progresses, it carries forward the top sequences (based on a predefined beam width, for example, 3) at each step. For instance, if "A" leads to "cat", "The" leads to "adorable", and "In" leads to "the", the algorithm may carry forward these top sequences to the next step.

As the search continues, the algorithm balances between the likelihood of the sequences and the diversity of possible outputs, eventually leading to the generation of a caption such as "A cat sitting on the windowsill", which takes into account both the likelihood of the words and the overall context of the image.

In this example, beam search allows for the consideration of multiple possible word sequences, ultimately resulting in a more context-aware and potentially more accurate caption for the given image.

Greedy Search

On the other extreme we can do a greedy search

At each time step, we always output the token with the highest probability (Greedy)

	time steps				
	1	2	3	4	5
cold	0.1	0.1	0.45	0.15	0.1
<stop>	0.15	0.15	0.05	0.3	0.5
coffee	0.25	0.25	0.1	0.35	0.2
I	0.4	0.05	0.05	0.01	0.1
like	0.05	0.35	0.15	0.09	0.05
water	0.05	0.1	0.2	0.1	0.05
	I	like	cold	coffee	<stop>

$$p(w_1 = I) = 0.4$$
$$p(w_2 = like | w_1 = I) = 0.35$$
$$p(w_3 = cold | w_1, w_2) = 0.45$$
$$p(w_4 = coffee | w_1, w_2, w_3) = 0.35$$
$$p(w_5 = stop | w_1, w_2, w_3, w_4) = 0.5$$

Then the probability of the generated sequence is

$$p(w_5, w_1, w_2, w_3, w_4) = 0.5 \times 0.35 \times 0.45 \times 0.35 \times 0.4 = 0.011$$

Greedy Search

Some Limitations

Is this the most likely sequence? — No

What if we want to get a variety of sequences of the same length? — Not possible

If the starting token is the word "I", then it will always end up producing the same sequence: I like cold coffee — Yes

Let's break down **Beam Search** with a beam width (or beam shape) of 2, using the sequence and words you provided: "Coffee," "I," "like," and "cold."

Scenario Setup:

You have 4 words: "Coffee," "I," "like," and "cold," and you start your decoding with `<Go>`. The model has given you **maximum probabilities** for two words initially: "I" and "Coffee."

Beam Search Overview:

Beam search is a heuristic search algorithm used to find the best sequence by keeping track of the top k (beam width) most probable sequences at each step. With a beam width of 2, the algorithm will keep track of the top 2 sequences at each step, rather than just the single most probable sequence (which is what greedy search does).

Step-by-Step Explanation with Beam Width of 2:

1. **Step 1: Start with `<Go>`**
   - The model outputs the probabilities for all possible next words after `<Go>`. Let's assume the top two words are "I" and "Coffee" with the highest probabilities.
   - **Beam:**
     - Sequence 1: `<Go> I` (high probability)
     - Sequence 2: `<Go> Coffee` (high probability)
   - These are the two most probable sequences at this step.
2. **Step 2: Expand Each Beam Sequence**
   - Now, the model predicts the next word for each of the sequences in the beam (`<Go> I` and `<Go> Coffee`).
   - Let's say for the sequence `<Go> I`, the model assigns high probabilities to "like" and "Coffee."
   - For the sequence `<Go> Coffee`, it assigns high probabilities to "like" and "cold."
   - For `<Go> I`:
     - Sequence 1.1: `<Go> I like`
     - Sequence 1.2: `<Go> I Coffee`
   - For `<Go> Coffee`:
     - Sequence 2.1: `<Go> Coffee like`
     - Sequence 2.2: `<Go> Coffee cold`
3. **Step 3: Prune Based on Combined Probabilities**
   - Now, we rank all the extended sequences from step 2 based on their cumulative probabilities.
   - For example:
     - `<Go> I like` might have the highest cumulative probability.
     - `<Go> Coffee like` might be next.
     - `<Go> I Coffee` and `<Go> Coffee cold` might have slightly lower probabilities.
   - Since the beam width is 2, we only keep the top 2 sequences based on their cumulative probabilities.
   - **Beam:**
     - Sequence 1: `<Go> I like`
     - Sequence 2: `<Go> Coffee like`
4. **Step 4: Expand the Remaining Sequences Again**
   - For the remaining sequences in the beam, we again predict the next word.
   - For `<Go> I like`, let's say the next highest probabilities are for "coffee" and "cold."
   - For `<Go> Coffee like`, the next highest probabilities are also for "cold" and "coffee."
   - For `<Go> I like`:
     - Sequence 1.1: `<Go> I like cold`
     - Sequence 1.2: `<Go> I like coffee`
   - For `<Go> Coffee like`:
     - Sequence 2.1: `<Go> Coffee like cold`
     - Sequence 2.2: `<Go> Coffee like coffee`
5. **Step 5: Final Pruning**
   - Finally, the algorithm ranks the extended sequences based on cumulative probabilities once more.
   - Let's assume the top two sequences are:
     - `<Go> I like coffee`
     - `<Go> Coffee like cold`
   - These are your final two candidate sequences after beam search with beam width 2.

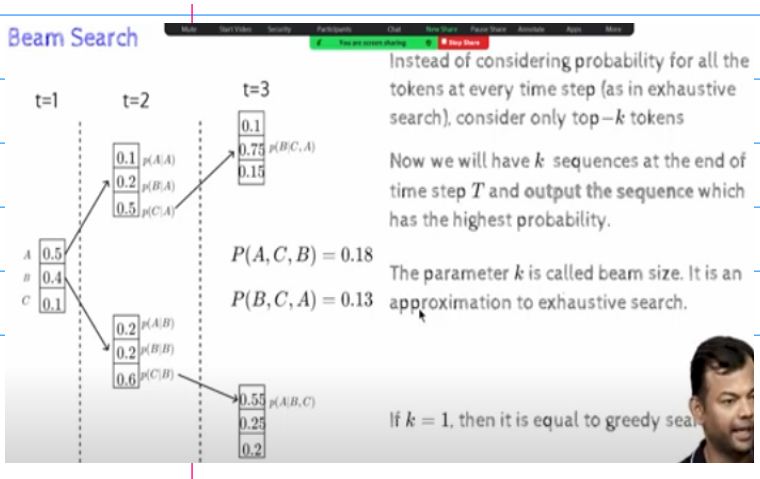
Final Result:

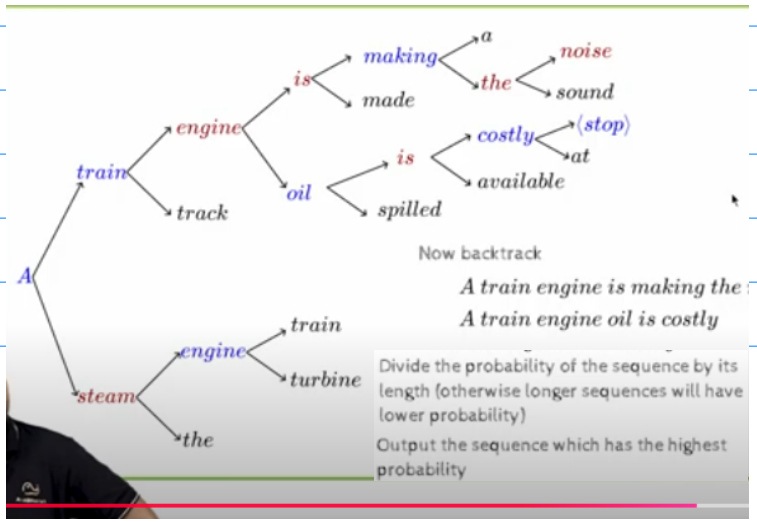
The beam search with beam width 2 will give you the top two most probable sequences based on the model's predictions and cumulative probabilities at each step. In this example, the top two sequences might be:

1. `<Go> I like coffee`
2. `<Go> Coffee like cold`

Summary of Beam Search with Beam Width 2:

- At each step, the algorithm expands each sequence in the beam by considering all possible next words.
- It keeps track of the top 2 sequences with the highest cumulative probabilities (since beam width = 2).
- The search continues until the desired sequence length is reached or all beams are exhausted.
- The final output is the most probable sequence(s), taking into account more than just the single most probable word at each step.





Both the greedy search and the beam search are prone to be degenerative

Latency for greedy search is lower than beam search

Neither greedy search nor beam search can result in creative outputs

Note however that the beam search strategy is highly suitable for tasks like translation and summarization

We are surprised when something is creative!

Surprise ∝ uncertainity

Top-K Sampling

Top-K sampling introduces non-deterministic outputs by focusing on selecting from the top probable choices for controlled randomness. Sampling from a distribution is used to determine token output, introducing randomness for creativity and surprise in outputs. Understanding sampling strategies in Transformer models involves generating probability distribution and normalizing it to facilitate controlled randomness.

An example of applying these takeaways could be using top-K sampling in a language model to select the top 3 probable words as potential continuations of a given sentence, thereby introducing controlled randomness and creativity to the generated text.

Example of Top-K Sampling with K = 3:

Let's say we're generating text starting with the prompt "**I enjoy**", and we are applying Top-K sampling with **K=3**.

Step 1: Model generates a probability distribution for the next word.

After the prompt "**I enjoy**", the model outputs a list of probabilities for possible next words:

- **eating**: 0.4
- **reading**: 0.35
- **traveling**: 0.2
- **sleeping**: 0.03
- **working**: 0.02

Step 2: Apply Top-K Sampling (K=3).

With **K=3**, we consider only the top 3 most probable words:

- **eating** (0.4)
- **reading** (0.35)
- **traveling** (0.2)

Now, we ignore lower-probability words like "sleeping" and "working" because their probabilities fall outside the top 3 choices.

Step 3: Normalize probabilities.

After applying Top-K, we renormalize the probabilities of the remaining 3 words so they sum to 1:

- **eating**: 0.4 / (0.4 + 0.35 + 0.2) = 0.421
- **reading**: 0.35 / (0.4 + 0.35 + 0.2) = 0.368
- **traveling**: 0.2 / (0.4 + 0.35 + 0.2) = 0.211

Step 4: Sample from the top 3 choices.

The model now randomly selects one of these 3 words, with the adjusted probabilities determining how likely each is to be chosen. Let's say it selects "**reading**" (based on the probabilities).

Generated Output So Far:

- "I enjoy reading"

Step 5: Repeat the process for the next token.

The process repeats for the next word. If the model generates probabilities for the next token, it will apply Top-K sampling again, choosing from the top 3 words at each step.

Result:

Using Top-K sampling with K=3 ensures that only the top 3 most probable words are considered at each step, introducing controlled randomness. This approach can generate creative and diverse outputs while maintaining a high likelihood of producing reasonable sentences.

Why Top-K Sampling is Useful:

- **Creativity**: It doesn't always pick the most probable word, which can lead to more varied and creative text.
- **Controlled randomness**: By limiting choices to the top K, it avoids extreme randomness, unlike pure sampling from the full probability distribution, while still allowing for flexibility in output generation.

Top-P Sampling (a.k.a. **Nucleus Sampling**):

Top-P sampling is a technique used in language generation that dynamically selects tokens from the **smallest set of top tokens whose cumulative probability mass exceeds a threshold P**. Unlike Top-K sampling, which fixes the number of tokens considered (K), Top-P sampling adapts based on the distribution of probabilities, allowing for a more flexible and dynamic token selection.

How It Works:

- Instead of selecting only the top K tokens, top-P sampling considers the top tokens whose cumulative probability exceeds a threshold P (e.g., P = 0.9).
- This allows the model to focus on the high-probability tokens while also allowing for more flexibility when the distribution is less certain (wider spread).
- **If the model is confident (has a few high-probability tokens), fewer tokens will be considered.** Conversely, if there is uncertainty (many low-probability tokens), more tokens will be considered to introduce variety.

Example of Top-P Sampling with P = 0.9:

Let's take the same scenario of text generation with the prompt "**I enjoy**" and look at the probability distribution generated for possible next words:

- **eating**: 0.4
- **reading**: 0.35
- **traveling**: 0.15
- **working**: 0.07
- **sleeping**: 0.03

Step 1: Compute the cumulative probability.

We sort the words by probability and sum them until the cumulative probability exceeds P = 0.9:

- **eating**: 0.4 (cumulative: 0.4)
- **reading**: 0.35 (cumulative: 0.75)
- **traveling**: 0.15 (cumulative: 0.9)

At this point, the cumulative probability has reached 0.9, so the set of possible tokens includes **eating**, **reading**, and **traveling**.

Step 2: Sample from the selected tokens.

Now, a token is randomly chosen from the set of **eating**, **reading**, and **traveling**, weighted by their probabilities. (Normalise and use randn(0,1)

eating reading traveling

0.45 0.38 0.17

randn(0,1) — 0.5 ∴ reading will be selected.

Let's assume that "**reading**" is selected.

Generated Output So Far:

- "I enjoy reading"

Step 3: Continue the process.

The same process is repeated for the next word. After generating "reading," the model would output the next probability distribution, and top-P sampling would dynamically adjust which tokens to consider based on their cumulative probabilities.

Example Use Case:

In a **chatbot application**, top-P sampling could be used with P = 0.9 to strike a balance between creativity and coherence. When the chatbot is responding to straightforward questions (high confidence), it selects from a smaller set of probable responses. In more open-ended conversations, it considers more varied responses, making interactions more engaging and less repetitive.

Key Benefits of Top-P Sampling:

1. **Dynamic Token Selection**: Top-P sampling is adaptive and flexible. It considers **as many tokens as needed** to reach a probability mass of **P**, allowing for more varied and creative outputs.
2. **Controlled Randomness**: By adjusting **P**, you can control the diversity of the output. A lower **P** (e.g., 0.7) will only allow very high-probability tokens (more deterministic output), while a higher **P** (e.g., 0.95) will include more tokens (more random output).
3. **Handles Uncertainty**: If the model is confident in its next prediction (few high-probability tokens), top-P sampling naturally limits the number of tokens to choose from. When the model is less certain, it considers more tokens, ei ing better handling of uncertainty.

Summary:

- **Top-P sampling** provides dynamic token selection based on cumulative probability, offering more flexibility than fixed Top-K sampling.
- It adjusts to the probability distribution at each step, allowing for controlled randomness and creativity in the generated text.

BERT

Introduction to BERT (Bidirectional Encoder Representations from Transformers):

BERT, introduced by **Google in 2018** (not 2017), revolutionized natural language processing (NLP) by leveraging a **bidirectional understanding of language** using the **Transformer** architecture. BERT significantly improved the performance of models on various NLP tasks such as **question answering**, **sentiment analysis**, and **named entity recognition**.

Unlike earlier models like GPT (which is **unidirectional** and reads the text left-to-right), **BERT processes text bidirectionally**, understanding the context of a word based on the words **before and after it**. This allowed BERT to capture richer information from both sides of a word, making it particularly effective for tasks that require deep contextual understanding.

1. **Bidirectional Representations in BERT**:

Traditional language models like GPT or LSTMs generate words in a unidirectional manner, either left-to-right or right-to-left. However, **BERT uses bidirectional context**, which means it looks at **both sides of the word** being predicted. For example, in the sentence "**The dog barked loudly**", when predicting "**dog**", BERT looks at both "**The**" (preceding) and "**barked loudly**" (succeeding).

2. **Masked Language Modeling (MLM)**:

BERT introduces a unique **pre-training objective** called **Masked Language Modeling (MLM)**. In this approach, 15% of the words in a sentence are randomly masked, and BERT is trained to **predict these masked words** based on the context provided by the surrounding words. This is different from other models, which typically predict the next word in sequence (causal language modeling). The **key idea behind MLM** is that it allows BERT to learn **deep bidirectional representations** by looking at the full context (both left and right) around the masked word.

How MLM Works:

- Given a sentence like "I enjoy [MASK] coffee in the morning", BERT's goal is to predict the masked word, which is "**drinking**".
- **15% of the words in the input** are masked during training.
- BERT processes the entire input sentence using a **stack of encoder layers** and predicts the masked words using the contextual information from both directions.

3. **Encoder-Only Architecture**:

BERT uses the **encoder** part of the **Transformer architecture**. The encoder consists of **multiple layers** (usually 12 or 16) of **multi-head self-attention** and **feed-forward neural networks**. Each layer outputs a contextualized representation of the input tokens.

- **Multi-Head Attention**: This mechanism allows the model to focus on different parts of the sentence simultaneously, capturing different relationships between words.
- **Feed Forward Networks**: After applying attention, the model uses a simple feed-forward neural network to process the information further.

4. **BERT's Objective Functions**:

BERT is pre-trained using two objectives:

1. **Masked Language Modeling (MLM)**: As explained above, 15% of input tokens are masked and BERT predicts the original tokens.
2. **Next Sentence Prediction (NSP)**: This task helps BERT understand sentence relationships. The model is given two sentences and asked to predict if the second sentence follows the first. This improves BERT's performance in tasks like **question answering** and **sentence pair classification**.

5. **Independence of Predictions in MLM**:

In MLM, each masked token's prediction is treated as **independent** of the other masked tokens. This means BERT does not sequentially predict tokens as in a generative model like GPT. Instead, it tries to predict each masked word based on the **context provided by all other non-masked words**.

For example, if the sentence is "I [MASK] coffee [MASK] the morning", the model predicts both masked words (e.g., "**enjoy**" and "**in**") independently based on the context from the unmasked words "I", "coffee", and "the".

6. **Prediction and Real Input Adaptation**:

In BERT's training, not all words are masked. **10% of the words are not masked** to better reflect real-world usage where words are not typically missing. This ensures that BERT can adapt to downstream tasks that use real (non-masked) inputs without degrading performance.

7. **Fine-Tuning BERT for Downstream Tasks**:

After pre-training on large corpora using MLM and NSP, BERT can be fine-tuned for specific downstream tasks (e.g., text classification, question answering, etc.). During fine-tuning, the entire model (including pre-trained layers) is updated based on the specific task's data.

- Fine-tuning usually requires **supervised data** for the task in question, such as labeled text.
- The **masking is not applied** during fine-tuning, as the task-specific data is used to directly train the model.

8. **Masked Attention in BERT**:

BERT's **attention mechanism** allows the model to selectively focus on certain parts of the input when making predictions. The attention weights are adjusted so that the model can "attend" to words in both directions (before and after the masked token). This helps in **recovering missing words** and predicting masked tokens by learning dependencies between distant words.

9. **Masked Tokens as Input**:

In BERT, masked tokens are represented by a special token, `[MASK]`. The model learns to predict these masked tokens by understanding their surrounding context. If we zero out the attention weights for these masked tokens, it would defeat the purpose of MLM, as the model would treat the masked tokens as non-existent.

10. **Architecture of BERT**:

BERT's architecture typically has **12 layers** (for **BERT-base**) or **16 layers** (for **BERT-large**), with **12 attention heads** per layer. The model is trained with 15% of input tokens masked, and a **probability distribution** is predicted over the vocabulary to identify the correct masked token.

Key Features of BERT:

- **Bidirectional**: BERT considers both previous and following tokens when predicting a word, unlike unidirectional models like GPT.
- **Masked Language Modeling**: Helps BERT learn contextual relationships between words by corrupting input and training the model to predict missing words.
- **Next Sentence Prediction**: Helps BERT understand relationships between sentence pairs.

Why BERT is Important:

- **Bidirectional Context**: Most NLP tasks benefit from understanding the full context of a sentence, which BERT provides by considering both left and right contexts.
- **Pre-trained Representations**: BERT's pre-trained representations can be fine-tuned for various tasks, significantly reducing the amount of task-specific training data needed.
- **Versatility**: BERT has been fine-tuned to excel in many NLP tasks, including classification, named entity recognition, question answering, and more.

Conclusion:

BERT has become a foundational model in NLP, setting the stage for further advancements in Transformer-based models. Its ability to capture bidirectional context through masked language modeling has made it a preferred choice for many NLP applications, from machine translation to text summarization and question answering.

Understanding BERT's mechanisms—such as **masked language modeling**, **multi-head attention**, and **pre-training/fine-tuning strategies**—is key to leveraging its power for a wide range of NLP tasks.

~~*Pre-training prepares models for downstream tasks.*~~
Feature extraction with pre-trained models in downstream tasks can be beneficial.
Using BERT as a feature extractor for downstream tasks is effective.
~~*Adapting pre-trained BERT model to downstream tasks is important.*~~
Pre-processing or feature extraction is done using fine-tuning.
Fine-tuning the entire BERT model for downstream tasks is a key aspect.
~~*Diverse data sets are essential for effective learning in machine learning models.*~~
Modeling the task of question answering using start and end token prediction is an important approach.
Identifying start and end tokens for valid spans is crucial for certain tasks.

GAS

Consider a vocabulary $\mathcal{V}$,

$\mathcal{V}$ =([start], building, character, a, is, astronomy, science, experience, natural,[end]).

Assume that we have a pre-trained GPT model for text generation and the first input token to the model is always the special token [start]. The prediction probabilities outputted by the model are given below.

$$\hat{Y} = \begin{bmatrix} 0.05 & 0.02 & 0.14 & 0.07 & 0.09 & 0.41 & 0.08 & 0.05 & 0.08 & 0.01 \\ 0.10 & 0.04 & 0.06 & 0.16 & 0.43 & 0.01 & 0.06 & 0.09 & 0.01 & 0.04 \\ 0.05 & 0.07 & 0.28 & 0.29 & 0.03 & 0.08 & 0.04 & 0.08 & 0.04 & 0.04 \\ 0.10 & 0.02 & 0.48 & 0.02 & 0.01 & 0.07 & 0.06 & 0.14 & 0.07 & 0.01 \\ 0.08 & 0.29 & 0.04 & 0.01 & 0.04 & 0.03 & 0.25 & 0.05 & 0.15 & 0.06 \\ 0.14 & 0.22 & 0.13 & 0.08 & 0.01 & 0.03 & 0.06 & 0.23 & 0.08 & 0.03 \end{bmatrix}$$

Start, Astr., is, a, char., building

The zeroth row of the matrix is the output probability distribution by the model given the special [Start] token as input. Following the Greedy Search decoding strategy, the subsequent rows give the conditional probability distribution conditioned over the previous tokens. If you think the given information is insufficient for any sub-questions, then enter -1 as your answer

1) What is the probability of the predicted word equal to ``Astronomy" given the ``[Start]" token as the initial input $P(y_1 = Astronomy | y_0 = [Start]$?

$P(y_1 = \text{Astronomy} \mid y_0 = \text{Start}) = 0.41$

2) What is the probability of the sequence ``Astronomy is a character building experience"?

0.23 0.29 0.48

$P(\text{Astronomy is a character building experience}) = P(\text{experience} \mid A, I, a, C, B) \times P(B \mid A, I, a, C) \times P(C \mid A, I, a)$
A I a C B

$\times P(a \mid A, I) \times P(I \mid A) \times P(A \mid \text{Start})$
0.29 0.43 0.41

$= 0.23 \times 0.29 \times 0.48 \times 0.29 \times 0.43 \times 0.41 = 0.0016.$

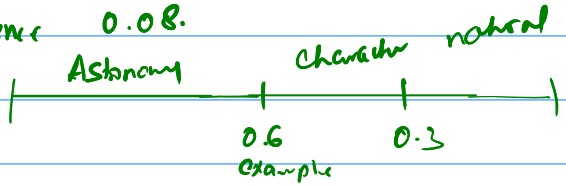
3) What is the probability of the sequence `` Astronomy is a natural science "?

0.41 → 0.43 → 0.29 → don't know — So Answer −1

4) Suppose we use Top$-k$ sampling, with $k = 3$, What is the probability of the predicted word equal to ``Astronomy" given the ``[Start]`` token as the initial input $P(y_1 = Astronomy | y_0 = [Start])$?

Start → Astronomy 0.41
Character 0.14
natural, science 0.08.

We normalize this and get



Example

x = rand (0, 1)

we don't know what is 'x', So Answer is −1

if $x < 0.6$, then 'Astronomy'

5) Let us consider the vocabulary $\mathcal{V}$ given below.

$\mathcal{V}$ =([CLS], building, character, a, is, astronomy, science, experience, natural,[SEP]).

Now we train the BERT model by randomly initializing the parameters. Suppose we feed in the following input sequence ``Astronomy is a [Mask] building [Mask] ", where we replaced the words ``character" and ``experience" with the special [Mask] token. We just feed this single sentence (segment) to the model and do Masked Language Modelling (MLP). Note that the special tokens [CLS] and [SEP] were not used. The matrix below gives the prediction probabilities for the given input sequence

$$\hat{Y} = \begin{bmatrix} 0.05 & 0.02 & 0.41 & 0.07 & 0.09 & 0.14 & 0.08 & 0.05 & 0.08 & 0.01 \\ 0.10 & 0.04 & 0.06 & 0.16 & 0.04 & 0.01 & 0.06 & 0.09 & 0.01 & 0.43 \\ 0.05 & 0.07 & 0.28 & 0.03 & 0.29 & 0.08 & 0.04 & 0.08 & 0.04 & 0.04 \\ 0.10 & 0.02 & 0.14 & 0.02 & 0.01 & 0.07 & 0.06 & 0.48 & 0.07 & 0.01 \\ 0.08 & 0.29 & 0.04 & 0.01 & 0.04 & 0.03 & 0.25 & 0.05 & 0.15 & 0.06 \\ 0.14 & 0.13 & 0.22 & 0.08 & 0.01 & 0.03 & 0.06 & 0.23 & 0.08 & 0.03 \end{bmatrix}$$

Astronomy, is, a, mask, build., mask.

$\therefore$ total loss $= \frac{1}{m} \Sigma - \ln \hat{y}$

$= \frac{1}{2} (-\ln 0.41 + (-\ln 0.23))$

$= \frac{1}{2} (1.96 + 1.47)$

$= \frac{1}{2} (3.43)$

$= 1.715$ ✓

What is the total loss value incurred by the model for the given input sequence? Use natural logarithm where required

6) Suppose we use the pre-trained BERT model for a sentiment classification problem. Assume that we use the model for feature extraction. That is, the model gives the representation for the given input sentence and we use that as input to some traditional classification algorithms. Then choose which of the following statements is (are) True

- [x] We can only take the representation of [CLS] token as an input to the classifier
- [ ] We can also use the representation of [SEP] token as an input to the classifier
- [ ] We can take representation of any word in the input sequence as an input to the classifier

"CLS" is a special token which gets info of all words.

Whereas if we use some 'word' or 'sep', they have more info about themselves rather than "all words".